

STANDARD OPERATING PROCEDURE

PROJECT 02 — PERSONALIZED RECOMMENDATION INTELLIGENCE PLATFORM

1. Project Objective

Build a production-style recommendation engine that learns from user interactions and generates personalized Top-K recommendations through candidate generation, retrieval and ranking.

2. Problem Statement

Users face too many choices in content platforms. The system should learn user preferences and recommend relevant items while providing measurable offline evaluation.

3. Expected System Flow

User Events → Data Processing → User/Item Representation → Candidate Generation → Candidate Retrieval → Ranking → Top-K Recommendations → Evaluation → FastAPI → Web Dashboard

4. Technology Stack

- Python, Pandas, NumPy and Scikit-learn.
- MovieLens dataset for initial development.
- Popularity baseline and collaborative filtering.
- User/item similarity and embeddings where appropriate.
- FastAPI for recommendation serving.
- Modern responsive frontend.
- Git/GitHub, .env configuration, logging and testing.

5. Dataset & Data Pipeline

- Use real MovieLens ratings/interactions, movie metadata, genres and timestamps.
- Validate users, items, ratings and timestamps.
- Create reproducible train/validation/test splits.
- Avoid temporal and user-information leakage during evaluation.
- Keep data loading/preprocessing separate from recommendation logic.

6. Recommendation Engine

- Implement a popularity-based baseline.
- Implement user-based and/or item-based collaborative filtering.
- Create user and item representations or embeddings where justified.
- Build candidate generation that produces a meaningful candidate pool.
- Rank candidates and return Top-K results.
- Keep recommendation strategies modular for future model additions.
- Return recommendation scores and transparent recommendation signals where possible.

7. Evaluation Requirements

- Calculate Precision@K, Recall@K, NDCG@K and Hit Rate.
- Measure catalog coverage where useful.
- Use a proper held-out evaluation protocol.
- Compare the final approach against the popularity baseline.
- Report only metrics calculated from the real system.

8. FastAPI Requirements

Endpoint	Purpose
GET /recommend/{user_id}	Return personalized Top-K recommendations.
GET /movies	Browse/search available items.
POST /feedback	Accept user interaction/feedback events.
GET /metrics	Return current evaluation metrics.
GET /health	Return service health status.

9. UI/UX Requirements

- Build a polished AI recommendation product, not a basic CRUD interface.
- Use an Antigravity-inspired modern visual language with clean typography, premium cards, subtle gradients and smooth micro-interactions.
- Responsive desktop/mobile layout.
- Home: personalized recommendations.
- Explore: search/filtering and item details.
- User Profile: history and interaction information.
- Recommendation Analytics: recommendation behaviour and useful system insights.
- Model Evaluation: real Precision@K, Recall@K, NDCG@K, Hit Rate and coverage.
- System Architecture: explain the recommendation pipeline.
- All interactions must connect to real backend functionality.

10. Security & Reliability

- Validate user IDs and API payloads.
- Use .env for configurable secrets and infrastructure values.
- Add error handling, logging and sensible timeouts.
- Design the API so rate limiting can be added later.
- Never expose credentials or internal stack traces.

11. Testing

- Test data processing and train/test splitting.
- Test recommendation generation for valid and unknown users.
- Test ranking and Top-K output.
- Test evaluation metrics independently.
- Test all API endpoints and frontend-to-API flows.
- Verify responsive UI, empty states and error handling.

12. Definition of Done

- Real MovieLens data produces real recommendations.
- Candidate generation and ranking are genuinely implemented.
- Evaluation metrics are calculated from held-out data.
- Frontend → API → recommendation engine works end-to-end.
- README, setup instructions, .env.example and API documentation are complete.
- No fake recommendations, fake metrics, dead buttons or placeholder pages remain.
- Project is clean, reproducible and deployment-ready.

Final Portfolio Standard: The project must demonstrate real recommender-system concepts, measurable evaluation and production-oriented engineering—not just a movie-similarity demo.